Technical Report: Microservices Orchestration on Amazon EKS

Prepared by: Yagayya Vig, Shrey Dhand
Date: April 2, 2026
Project: ACIT 3495 – Video Streaming System (Cloud Migration)

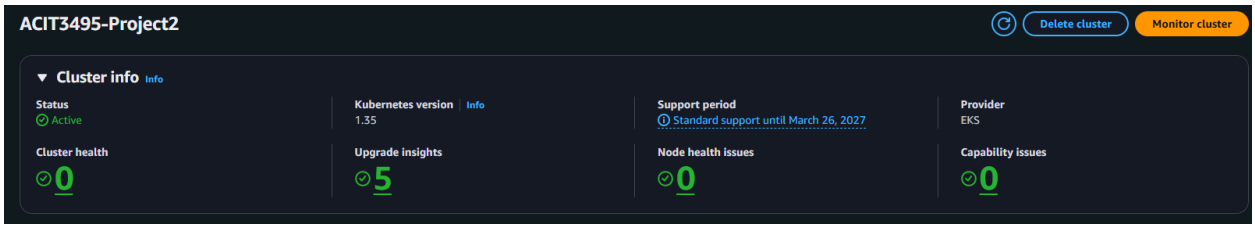
1. Executive Summary

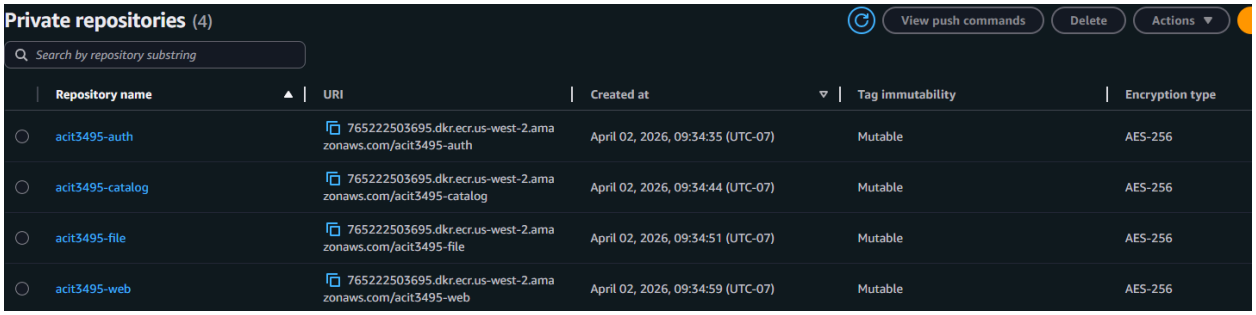
The objective of this project was to migrate a localized microservices-based video streaming application to a production-grade cloud environment. Using **Amazon Elastic Kubernetes Service (EKS)** and **Amazon Elastic Container Registry (ECR)**, the system was architected to ensure high availability, public accessibility, and dynamic horizontal scaling.

2. Infrastructure & Containerization

The foundation of the deployment rests on a decoupled architecture where each service is isolated within its own container.

- **Registry Strategy:** Four private repositories were established in Amazon ECR to host the `auth`, `catalog`, `file-service`, and `web` images.
- **Cluster Configuration:** An EKS cluster, `ACIT3495-Project2`, was deployed in the `us-west-2` region. **EKS Auto Mode** was utilized to manage the underlying compute resources (EC2 nodes) automatically.





Description: This screenshot confirms the successful creation of the four service repositories and the presence of the latest image tags.

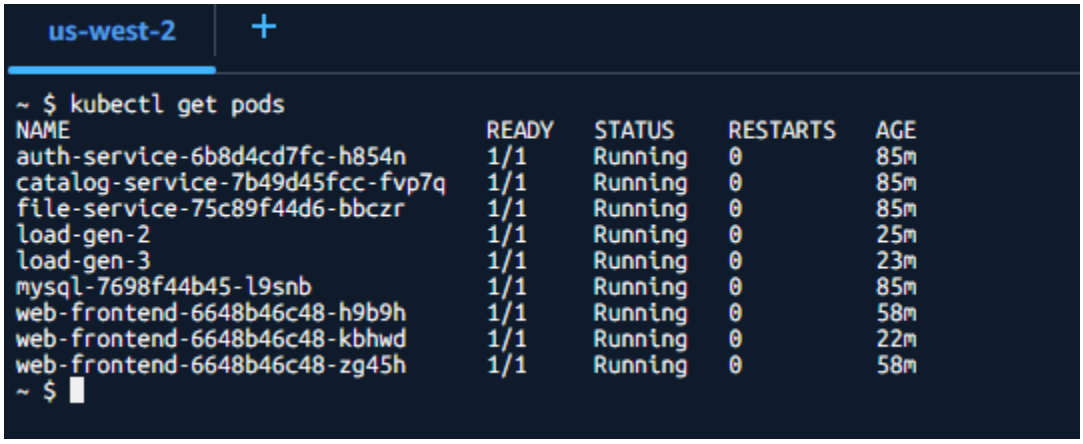
---

3. Deployment & Orchestration

The application was deployed using a declarative approach with Kubernetes manifests.

- **Stateful Services:** A MySQL 8.0 database was deployed using a `ClusterIP` service and a `ConfigMap` to handle schema initialization.
- **Microservices:** Each functional unit was deployed with a specific number of replicas to ensure redundancy.

Service	Type	Port	Purpose
Auth	LoadBalancer	4000	Identity & Token Management
Catalog	LoadBalancer	5001	Metadata & Video Discovery
File	LoadBalancer	5000	Content Storage & Retrieval
Web	LoadBalancer	80	Public Entry Point/UI



Description: Validation showing all application pods and the database in a "Running" state.

4. Networking & Connectivity

Public access was achieved by provisioning **AWS Network Load Balancers (NLB)**. To facilitate this, the VPC subnets were tagged with `kubernetes.io/role/elb = 1`, enabling the EKS Load Balancer Controller to dynamically assign public IPs to the services.

```
~ $ kubectl get svc
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP                                                                    PORT(S)          AGE
auth-service      LoadBalancer   10.100.235.178   abd56e60f027a4068ad133f9aa7e27e0-151086203.us-west-2.elb.amazonaws.com         4000:30645/TCP   86m
catalog-service   LoadBalancer   10.100.183.247   a37532e57d2754894be9787b2422a15d-1612828023.us-west-2.elb.amazonaws.com        5001:31525/TCP   86m
file-service      LoadBalancer   10.100.38.63     a1e9f6ca8ec544b05b0d21518d019097-1988058299.us-west-2.elb.amazonaws.com        5000:31860/TCP   86m
kubernetes        ClusterIP      10.100.0.1       <none>                                                                         443/TCP          93m
mysql             ClusterIP      10.100.37.114    <none>                                                                         3306/TCP         86m
web-service       LoadBalancer   10.100.248.206   k8s-default-webservi-290d27dab3-ca61fe047f68e271.elb.us-west-2.amazonaws.com   80:30747/TCP     66m
~ $
```

Description: Listing of the External-IP URLs for each microservice.

5. Dynamic Scalability Analysis (HPA)

A core requirement of the system was the ability to handle traffic spikes. A **Horizontal Pod Autoscaler (HPA)** was configured for the `web-frontend` deployment.

- **Metric:** CPU Utilization.
- **Threshold:** 50% (temporarily lowered to 10% for validation).
- **Range:** 2 (Min) to 10 (Max) Replicas.

During the stress test, the system detected increased load and automatically scaled the frontend from 2 to 10 pods within minutes. Once the load was removed, the cluster successfully executed a "cool-down" period and returned to the minimum replica count.

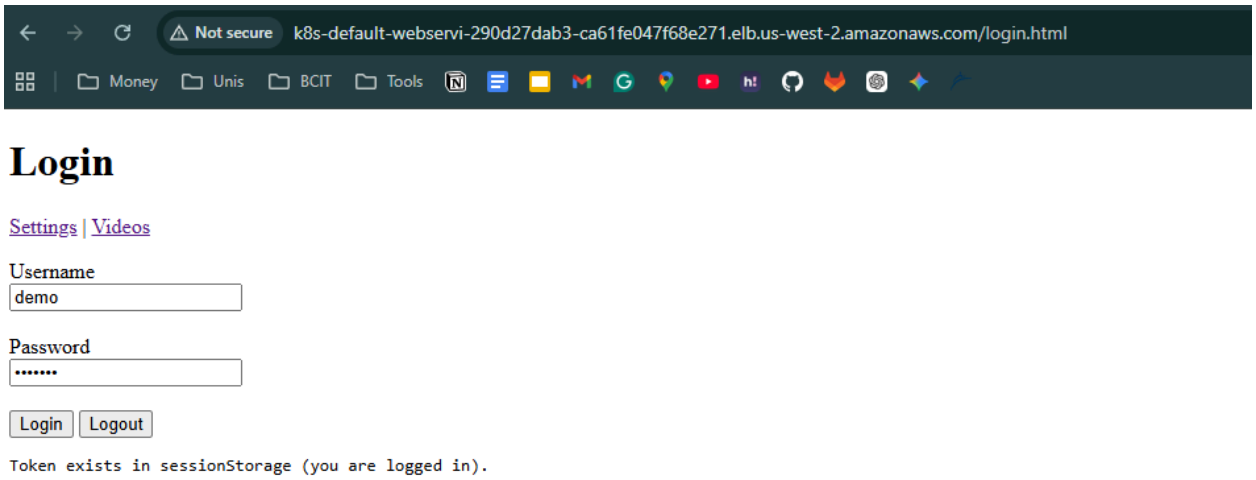
```
~ $ kubectl get svc
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP                                                                    PORT(S)          AGE
auth-service      LoadBalancer   10.100.235.178   abd56e60f027a4068ad133f9aa7e27e0-151086203.us-west-2.elb.amazonaws.com         4000:30645/TCP   86m
catalog-service   LoadBalancer   10.100.183.247   a37532e57d2754894be9787b2422a15d-1612828023.us-west-2.elb.amazonaws.com        5001:31525/TCP   86m
file-service      LoadBalancer   10.100.38.63     a1e9f6ca8ec544b05b0d21518d019097-1988058299.us-west-2.elb.amazonaws.com        5000:31860/TCP   86m
kubernetes        ClusterIP      10.100.0.1       <none>                                                                         443/TCP          93m
mysql             ClusterIP      10.100.37.114    <none>                                                                         3306/TCP         86m
web-service       LoadBalancer   10.100.248.206   k8s-default-webservi-290d27dab3-ca61fe047f68e271.elb.us-west-2.amazonaws.com   80:30747/TCP     66m
~ $
```

Description: Proof of the system reaching maximum capacity during a simulated traffic surge.

6. Functional Validation

The final phase of the project confirmed the end-to-end integration of the frontend and backend services. By configuring the Web UI with the public Load Balancer URLs, the following features were verified:

- User Authentication (JWT Token generation).
- Database Retrieval (Video list population).
- Public UI rendering.



Description: The functional web interface running on the public AWS URL.

---

7. Conclusion

The migration was successful. The system effectively leverages AWS managed services to provide a scalable, resilient, and accessible platform for video streaming. All project requirements, including containerization, cloud orchestration, and automated scaling, were met and verified.